

Лабораторийн ажил

(Лаборатори №2)

МУИС, МТЭС, МКУТ-ийн оюутан: Н. Мөнхпүрэв /SiSi ID: 23B1NUM0114/

1. ОРШИЛ

Энэхүү лабораторийн ажлаар C++ хэлэнд ашиглагддаг санах ойн хаяг, хаяган хувьсагч болон заалтан хувьсагчийн үндсэн ойлголтуудыг судалж, эдгээрийг хэрхэн ашиглах талаар практик дадлага хийлээ. Лабораторийн явцад дараах сэдвүүдийг хамарсан:

- Хаяган хувьсагчийн ажиллах зарчим □
- Заалтан хувьсагчийн ажиллах зарчим
- Заалтан хувьсагч болон хаяган хувьсагчийн ялгаа

2. ЗОРИЛГО

- Хаяган хувьсагчийг ашиглан санах ойн хаягийг хадгалах, утгуудыг өөрчлөх
- Заалтан хувьсагчийг ашиглан хувьсагчийн утгыг шууд өөрчлөх
- Хаяган хувьсагч болон заалтан хувьсагчийн ялгааг ойлгох

3. ОНОЛЫН СУДАЛГАА

- “Хаяг бол тоон утга тул түүнийг хувьсагч руу хадгалж болох ба хаяг хадгалах хувьсагчийг хаяган хувьсагч гэнэ.”ⁱ Хаягийн утгыг авахын тулд * оператор хэрэглэнэ.
- Заалтан хувьсагчийг тодорхой нэр бүхий хувьсагчтай холбож үүсгэнэ. Чингэж тухайн хувьсагчийн утгийг шууд өөрчлөх боломжийг олгодог. Энэ төрлийн хувьсагчид & оператороор хандаж, функцад параметрээр дамжуулж болно.

Шинж чанар	Хаяган хувьсагч (Pointer)	Заалтан хувьсагч (Reference)
Тодорхойлолт	Өгөгдлийн санах ойн хаягийг агуулсан хувьсагч	Өөр хувьсагчийн эсвэл өгөгдлийн өөр нэр, alias
Синтакс	int* p;	int& r = a;
Утга	Санах ойн хаяг хадгална	Утгыг өөр хувьсагчийн утгатай шууд холбодог
Шинэ утга оноох	Заагчийн утгыг өөрчилж болно (бусад хаяг руу заах боломжтой)	Нэгэнт үүссэн бол өөр өгөгдөл рүү зааж чадахгүй
Null байж болно	Тийм, NULL (nullptr) утга авч болно	Үгүй, заавал эх хувьсагчийг заах ёстой

Динамик өөрчлөлт	Заагчийг өөр хаяг руу шилжүүлж болно	Заалттай хувьсагч байнга тухайн нэг хувьсагчийг заана
Угсралт	Заагчийн санах ойн хаягийг зааж өгнө, *p-аар хандаж утга руу нэвтэрнэ	Зөвхөн өөр хувьсагчийн alias (өөр нэр) бөгөөд г гэдэг нэрээр шууд хандаж болно

Заалтан хувьсагч, Хаяган хувьсагчийн ялгаа тойм

4. ХЭРЭГЖҮҮЛЭЛТ

- а. Хаяган хувьсагч ашиглан хоёр хувьсагчийн утгийг солих функцийг хэрэгжүүлэхэд swap() функцийн түгээмэл хэрэглээ болох түр зуурын санах ой зарлан, утгуудыг сөөлжилж өгч шийдсэн.

```
void hayg_solih(int *tool, int* too2){
    int t = *tool;
    *tool = *too2;
    *too2 = t;
}
```

Хаяган хувьсагч ашиглан 2 хувьсагчийн утга солих функц

5. ДҮГНЭЛТ

Заалтан хувьсагч нь хялбар бөгөөд хурдан ажиллах боломжийг олгодог. Үүний тусламжтайгаар хувьсагчийн утгыг шууд өөрчлөж тоог эрэмбэлсэн.

Хаяган хувьсагч нь санах ойн хаягийг хадгалахад ашиглагддаг. Энэ нь хаяг дээр хадгалагдсан утгыг шууд удирдаж болно. Мөн үүний тусламжтай функцэд хаягийг дамжуулж түүгээр утгыг нь өөрчилсөн.

6. АШИГЛАСАН МАТЕРИАЛ

1. Объект хандлагат технологийн C++ програмчлал, Ж.Пүрэв, 2008, Улаанбаатар.

7. ХАВСРАЛТ

```
1.c++ 4.cpp x 5.cpp 2.cpp 3.cpp
Lab2 > 4.cpp > main()
1  #include<iostream>
2
3  using namespace std;
4  int main(){
5
6      int numbers[5];           // 5 shirheg int elemttei massive zarlaj baina.
7
8      int * p;                  // INT turliin p haygan huvisagch zarlav.
9
10     p = numbers; *p = 10;      // p numbers[0] ruu zaana. *p = 10 -> numbers[0] = 10;
11     p++; *p = 20;              // p numbers[1] ruu zaana. *p = 20
12
13     p = &numbers[2]; *p = 30;  // p numbers[2] ruu zaana. *p = 30;
14
15     p = numbers + 3; *p = 40;  // p numbers[3], *p = 40
16
17     p = numbers; *(p+4) = 50;  // p numbers[0], *(p + 4) -> numbers[4]; numbers[4] = 50
18
19     for (int n=0; n<5; n++)
20
21         cout << numbers[n] << ", "; // massive-iin buh elemntiig hevlene.
22
23     return 0;
24
25
26
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

10, 20, 30, 40, 50, [1] + Done "usr/bin/gdb" --interpreter=mi --tty=\${DbgTerm} 0<"/tmp/Micros

```
1.c++ 4.cpp 5.cpp 2.cpp x 3.cpp
Lab2 > 2.cpp > main()
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6
7
8      char *p1;                 // Char turliin p1 pointer zarllaa
9
10     int *p2;                   // int ruliin p2 pointer zarllaa
11
12     double *p3;               // DOUBLE turliin p3 huvisagch zarlav
13
14
15     cout<<sizeof(p1)<<sizeof(p2)<<sizeof(p3) << endl; //хэд хэд гэсэн утга хэвлэж байна вэ? Яагаад?
16     return 0;                 // pointeriin hemjee 64bit OS-d 8 bite baidag tul 888 gej hevlene
17
18
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

10, 20, 30, 40, 50, [1] + Done "usr/bin/gdb" --interpreter=mi --tty=\${DbgTerm} 0<"/tmp/Microsoft-MIEngine-In-n

```
1.c++ 5.cpp x
Lab2 > 5.cpp > ...
1  #include<iostream>
2  using namespace std;
3
4  void hayg_solih(int *tool, int* too2){
5      int t = *tool;
6      *tool = *too2;
7      *too2 = t;
8  }
9
10 int main(){
11
12     int i = 0, j = 1;
13     cout << "Solihoos umnu: i = " << i << ", j = " << j << endl;
14
15     hayg_solih(&i, &j);
16
17     cout << "Solisnii daraa: i = " << i << ", j = " << j << endl;
18
19     return 0;
20 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Solihoos umnu: i = 0, j = 1
Solisnii daraa: i = 1, j = 0
[1] + Done "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microsoft-MIEngine-In-y
soft-MIEngine-Out-ztqblyyx.goe"
```

```
1.c++ 4.cpp 5.cpp 2.cpp 3.cpp x
Lab2 > 3.cpp > ...
1 #include <iostream>
2 using namespace std;
3
4 int main(){
5
6     int a=125;           // a-d 125iig onoov
7
8     int *p = &a;         // int турлиин p pointert a-ийн haygiig onoov
9
10    cout<<p << endl;     // p-ийн utga buyu a-ийн haygiig hevlev
11
12    cout<<*p << endl;    // p pointerийн зааж буи utga buyu a-ийн utgiig hevlene
13
14    p++;                 // p pointerийг int хемжее buyu 4 byte(1-еер) nemegduulev
15
16    cout<<p << endl;     // nemegduulsen utga buyu 4 byte iluu
17
18    cout<<*p << endl;    // nemegduulsen haygandah utgiig hevlene
19    return 0;
20 }
21

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
10, 20, 30, 40, 50, [1] + Done "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microsoft-MIEngine-In-m
1.c++ x 2.cpp
Lab2 > 1.c++
1 // 1. Санах ойн хаяг гэж юу вэ? Хаягийн хэмжээ хэд байдаг вэ?
2 // Санах ой (Мемогу) гэдэг нь компьютерын өгөгдлийг түр хадгалах хэсэг юм.
3 // Жишээ нь: хувьсагч, функц, объект гэх мэт зүйлс санах ойд хадгалагддаг.
4 // Санах ой нь олон жижиг хэсгүүдээс (memory cell) бүрддэг.
5 // Тэдгээр хэсэг бүр нэг хаягтай байдаг.
6 // Санах ойн хаяг гэдэг нь тухайн санах ойн нэгжийн байршлыг заадаг тоо юм.
7 // Энэ хаяг нь тухайн мэдээллийг хаана хадгалж байгааг тодорхойлно.
8 // Хаягийн хэмжээ нь процессорын архитектураас хамаардаг.
9 // Энэ нь тухайн машин хэдэн битийн хаягийг ашиглаж байгаагаас хамаарна.
10 // Архитектур Хаягийн хэмжээ Тайлбар
11 // 32-bit 4 байт (32 бит)  $2^{32} = 4 \text{ GB}$  хүртэлх хаяг өгч чадна
12 // 64-bit 8 байт (64 бит)  $2^{64} = \text{асар их хэмжээний хаяглалт боломжтой}$ 
13
14 // Хувьсагчийн эзэмших ойн ангилал нь тухайн хувьсагч руу программын аль хэсгээс хандаж болох,
15 // уг хувьсагч хэр удаан устахгүй оршихыг заадаг байна.
16 //
17 //
18 //
19
```

```
1.c++ 6.cpp x
Lab2 > 6.cpp > main()
1  #include<iostream>
2  using namespace std;
3
4  void hayg_solih(int &too1, int& too2){
5      int t = too1;
6      too1 = too2;
7      too2 = t;
8  }
9
10 int main(){
11
12     int i = 0, j = 1;
13     cout << "Solihoos umnu: i = " << i << ", j = " << j << endl;
14
15     hayg_solih(i, j);
16
17     cout << "Solisnii daraa: i = " << i << ", j = " << j << endl;
18
19     return 0;
20 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Solihoos umnu: i = 0, j = 1
Solisnii daraa: i = 1, j = 0
[1] + Done "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microsoft-MIEngine-In-a
soft-MIEngine-Out-hoogjiirr.lgu"
```

ⁱ Объект хандлагат технологийн C++ програмчлал, Ж.Пүрэв, 2008, Улаанбаатар.